

Systems and Methods for Adaptive Mixed-Bit Quantization of Neural Network Weights Using Fixed-Size Macroblocks and GPU-Friendly Unpacking ("FP4 Hybrid")

Title

Systems and Methods for Adaptive Mixed-Bit Quantization of Neural Network Weights Using Fixed-Size Macroblocks and GPU-Friendly Unpacking

Technical Field

This disclosure relates to model compression and accelerated inference, and more particularly to encoding and decoding neural network parameters using low-bit quantization formats optimized for parallel processing hardware such as graphics processing units (GPUs) and other parallel compute devices.

Background

Neural network inference performance is often limited by memory bandwidth and cache behavior when accessing weight tensors. Low-bit quantization can reduce bandwidth but may introduce accuracy loss and or decode overhead. Variable-length compression schemes can improve density but often lead to irregular memory access patterns and increased control flow complexity, thereby reducing decode throughput on GPUs.

Accordingly, there is a need for a quantization format that provides high decode throughput on GPUs, supports adaptive precision where beneficial, and maintains predictable, aligned memory access.

Summary

Disclosed herein are systems and methods that encode a weight tensor into macroblocks and store each macroblock as a fixed-size payload with an associated affine header. In some embodiments, each macroblock corresponds to a 64-by-64 region of the weight tensor and is represented by a 2048-byte payload stored at an aligned address. The 2048-byte payload size enables uniform addressing and predictable memory access on GPUs.

In some embodiments, each macroblock includes:

- 1) a header containing an affine parameter set (scale and bias), and
- 2) a payload of exactly 2048 bytes containing packed codes representing the macroblock.

In some embodiments, the affine parameter set (scale and bias) is used for reconstruction for all supported code modes, such that decoded weights are computed as:

$$w_hat = S * code + Bias$$

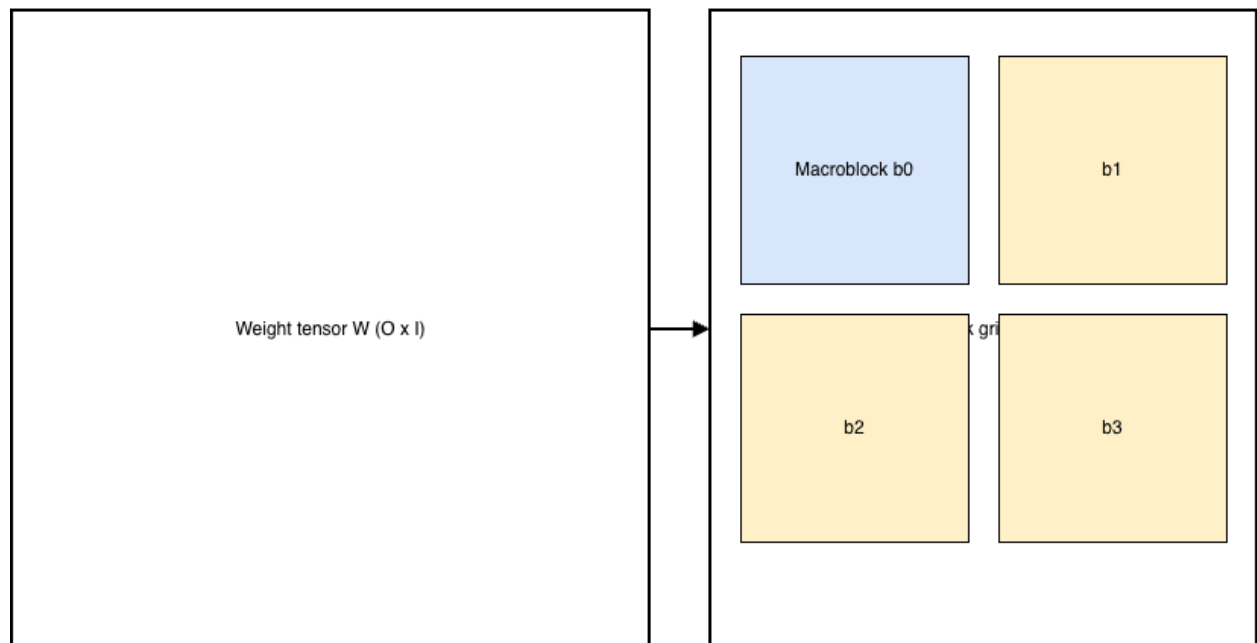
In some embodiments, the payload encodes codes according to a selected mode. Non-limiting examples of modes include:

- FP4_AFFINE: 4-bit signed codes, and
- T158_AFFINE: ternary codes (values in $\{-1, 0, +1\}$).

In some embodiments, a header array stores per-macroblock affine parameters, and an offsets array points into a codes blob containing concatenated 2048-byte payloads. In some embodiments, because payload offsets are aligned, the low bits of each offset word store mode flags, thereby reducing metadata bandwidth and improving decode throughput.

Brief Description of the Drawings

FIG. 1 Tiling a weight matrix into macroblocks



Each macroblock encodes 4096 weights
(64 x 64)

FIG. 2 Container layout with headers, offsets, and codes blob

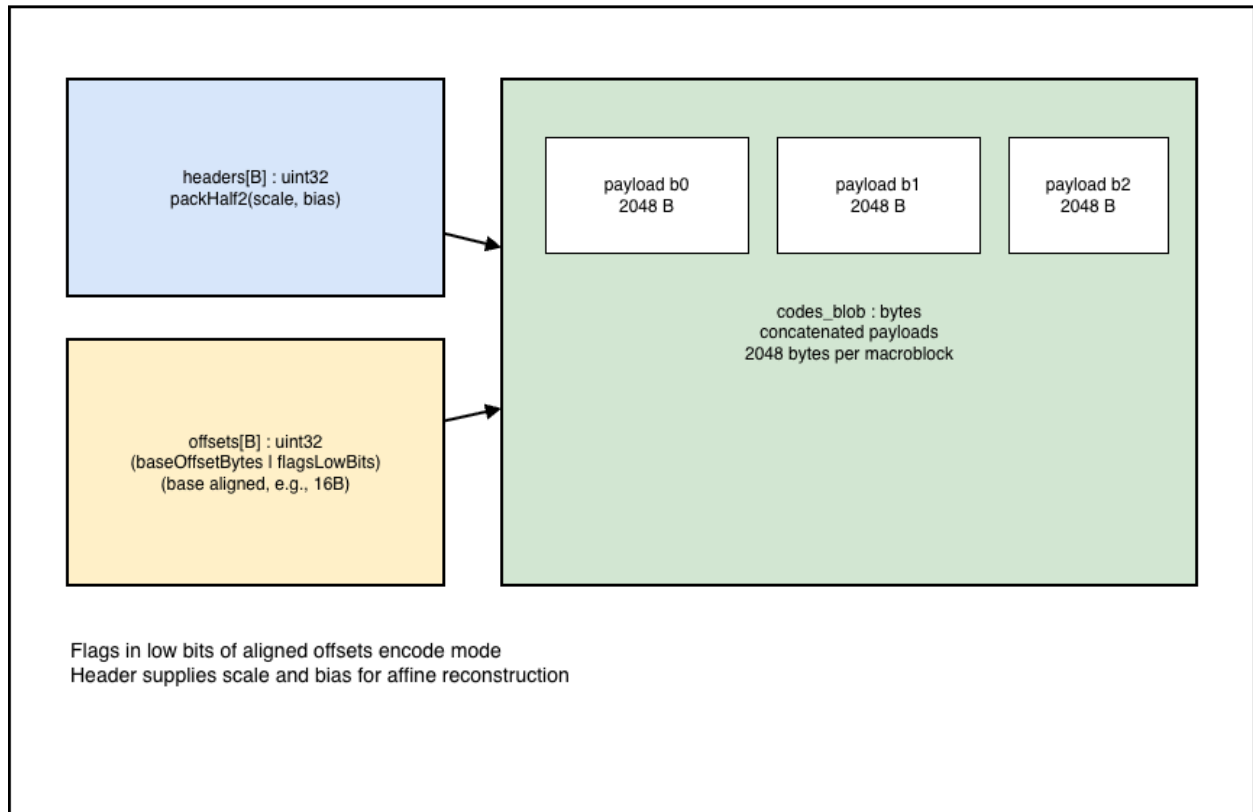


FIG. 3 Fixed-size payload formats (2048 bytes per macroblock)

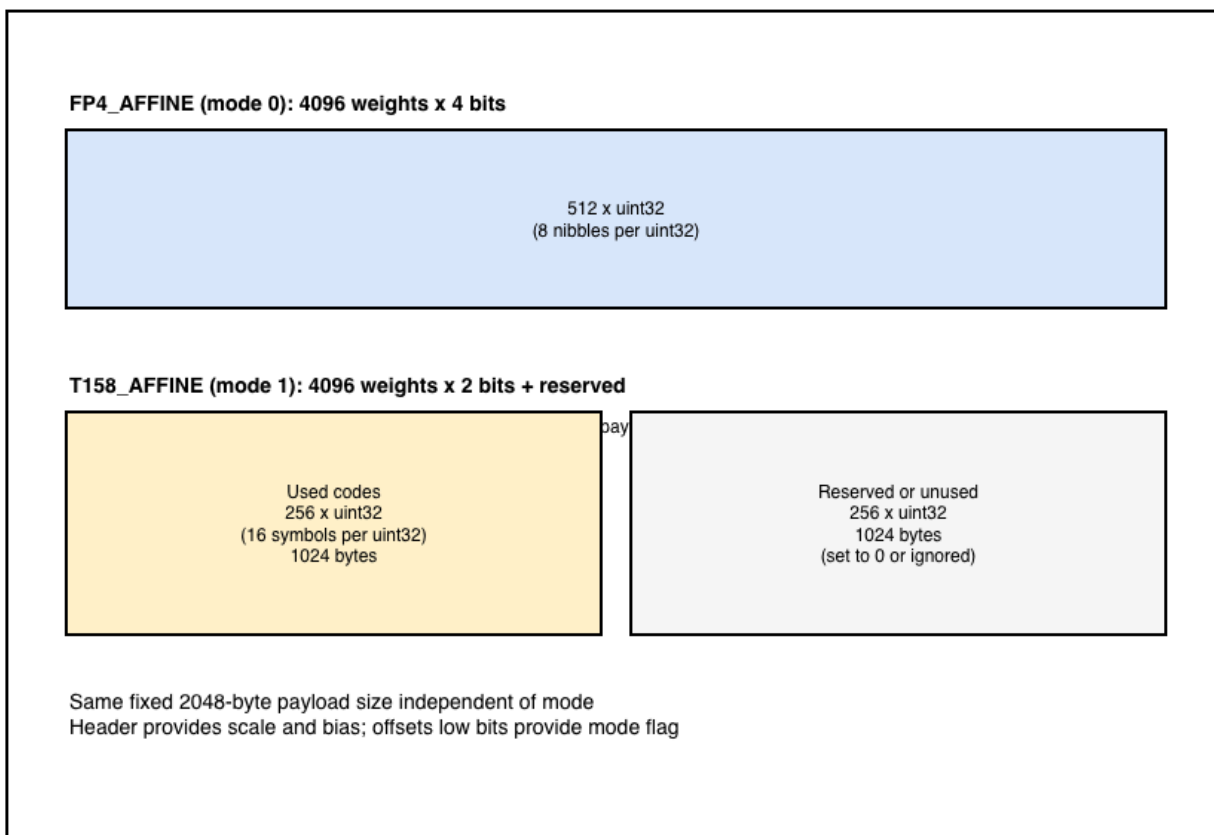
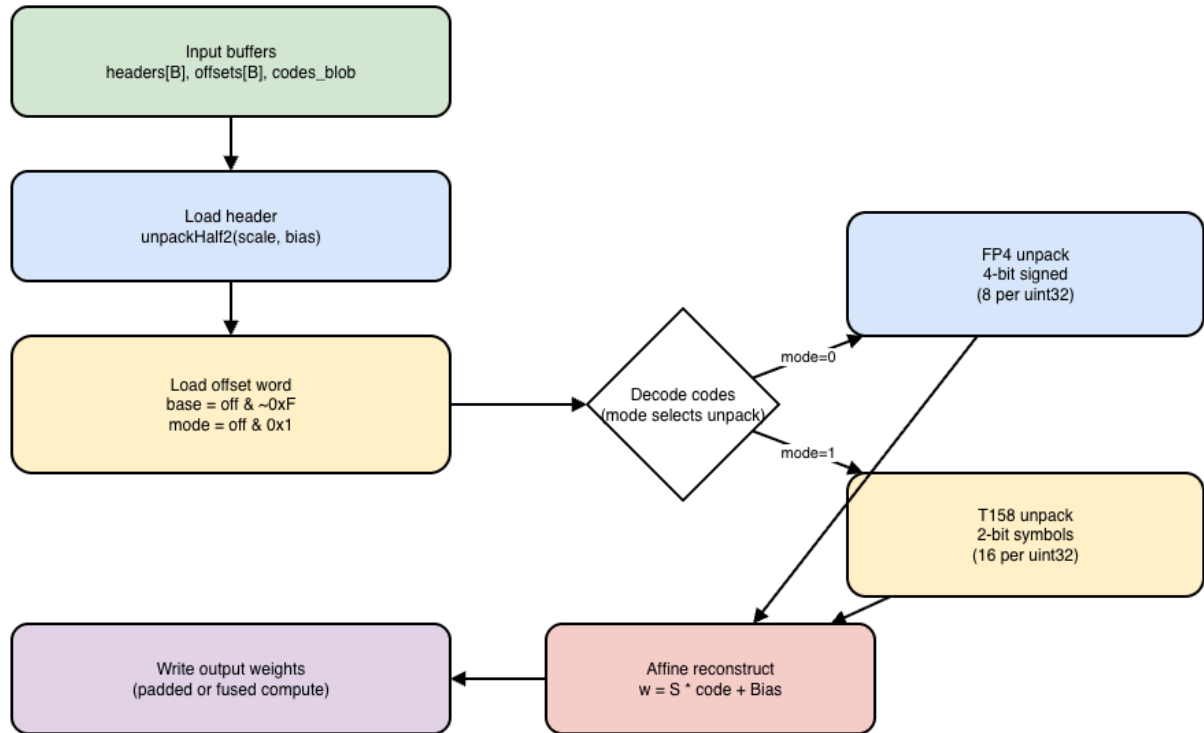


FIG. 4 GPU unpack and affine reconstruction flow



Detailed Description

1. Definitions

- Weight tensor W of shape $[O, I]$ (for example, output channels by input channels), processed as floating-point values during model execution.
- Macroblock: a block of weights of size M by N , such as 64 by 64 (4096 weights). Other sizes may be used.
- Affine decode: decoded value $w_hat = S * code + Bias$, where S is a per-block scale and $Bias$ is a per-block bias.
- Fixed per-block payload: a constant-size encoded representation for each macroblock, enabling uniform memory addressing and predictable cache behavior.

2. Macroblock tiling

Given W of shape $[O, I]$, compute macroblock grid dimensions based on a chosen macroblock size.

In a 64 by 64 embodiment:

- $tiles_y = \text{ceil}(O / 64)$

- `tiles_x = ceil(l / 64)`
- `total blocks B = tiles_y * tiles_x`

In some embodiments, `W` is padded to `(tiles_y * 64, tiles_x * 64)` and then divided into blocks indexed by:

- `by = b / tiles_x`
- `bx = b % tiles_x`
- `block(b) = W_padded[by*64:(by+1)*64, bx*64:(bx+1)*64]`

3. Container layout (headers, offsets, codes blob)

In some embodiments, the quantized tensor is stored using three logical arrays:

- 1) `headers[B] : uint32` (packed half2 containing scale and bias)
- 2) `offsets[B] : uint32` (byte offsets into codes blob, with low bits reserved as flags)
- 3) `codes_blob[] : bytes` (concatenated per-block payloads)

In some embodiments, each macroblock payload occupies 2048 bytes in `codes_blob`.

In some embodiments, the original tensor shape (`O`, `I`) and macroblock shape (`M`, `N`) are stored in a higher-level container, manifest, or model file metadata.

3.1 Payload alignment and embedded flags

In some embodiments, each 2048-byte payload begins at an address aligned to a predetermined boundary (for example, 16 bytes). Because aligned byte offsets have low bits that are always zero, the system stores mode and or metadata flags in these low bits:

`offsets[b] = (codesOffsetBytes & ~0xF) | flags4`

At decode:

- `flags4 = offsets[b] & 0xF`
- `baseBytes = offsets[b] & ~0xF`

In some embodiments, the flags include:

- bit 0: mode (0 = `FP4_AFFINE`, 1 = `T158_AFFINE`)
- bits 1 to 3: optional additional metadata or reserved

In some embodiments, embedding mode flags in the offsets array reduces the number of memory reads required to determine how to decode each payload.

4. Per-block header (scale and bias)

Each macroblock b has a header `headers[b]` packing two floating-point values:

- scale (S)
- bias ($Bias$)

In some embodiments, scale and bias are stored as half-precision floating point values packed into a single 32-bit word. During decode, S and $Bias$ are unpacked (e.g., using `unpackHalf2x16` in GLSL or an equivalent operation).

In some embodiments, all supported modes share the same header semantics and the same affine reconstruction formula:

$$w_hat = S * code + Bias$$

5. Fixed payload size (2048 bytes)

In some embodiments, each macroblock payload in `codes_blob` is exactly 2048 bytes (512 `uint32`). Fixed size reduces scatter, enables uniform addressing, and allows simple per-block base computation.

In some embodiments, 2048 bytes corresponds to storing 4096 weights at 4 bits each. In some embodiments, modes that use fewer bits per weight store their codes within a subset of the 2048 bytes, setting the remaining bytes to zero or leaving them unused.

6. Example quantization modes

6.1 Mode A: `FP4_AFFINE` (4-bit signed codes)

`FP4_AFFINE` encodes each weight as a signed 4-bit two's-complement code q in $[-8, 7]$.

Packing (non-limiting):

- 8 codes per `uint32` word, each code occupying 4 bits
- for 64 by 64: $4096 * 4 \text{ bits} = 2048 \text{ bytes}$

Decode per element:

- $code = q$
- $w_hat = S * q + Bias$

6.2 Mode B: `T158_AFFINE` (ternary codes stored as 2-bit symbols)

`T158_AFFINE` represents values in $\{-1, 0, +1\}$ and reconstructs using the same affine header (scale and bias).

In some embodiments, ternary values are stored as 2-bit symbols:

- 00 maps to 0
- 01 maps to +1
- 10 maps to -1
- 11 is reserved (may be decoded as 0)

Payload usage for a 64 by 64 macroblock (non-limiting):

- first 1024 bytes contain $4096 * 2$ bits
- remaining 1024 bytes are zero or unused to preserve the fixed 2048-byte payload size

Decode per element:

- $\text{code} = t \text{ in } \{-1, 0, +1\}$
- $\hat{w} = S * t + \text{Bias}$

7. Adaptive mode selection (optional but supported)

In some embodiments, an encoder selects a mode per macroblock based on an error metric.

In some embodiments:

- A first candidate reconstruction is generated for FP4_AFFINE.
- A second candidate reconstruction is generated for T158_AFFINE.
- An error metric is computed for each candidate and used to select a mode.

The error metric may be an L2 norm, an L1 norm, an infinity norm, a cosine distance, a task-weighted metric, or a pyramid-weighted metric.

8. Exemplary GPU unpacking compute shader code (GLSL-like)

The following is exemplary decode logic suitable for implementation in a GPU compute shader. Other GPU programming models (for example, HLSL, CUDA, Metal) may be used.

8.1 Helper functions

```
int int4_to_int(int nib) {
    nib &= 15;
    return (nib < 8) ? nib : (nib - 16);
}
```

```
int sym2_to_ternary(int sym) {
    sym &= 3;
    if (sym == 0) return 0;
    if (sym == 1) return 1;
    if (sym == 2) return -1;
    return 0;
}
```

8.2 Kernel outline (per-macroblock decode)

```
layout(local_size_x = 256) in;
```

```
void main() {
    uint b = gl_WorkGroupID.x;
    uint lane = gl_LocalInvocationID.x;
    uint h = headers[b];
    vec2 sb = unpackHalf2x16(h);
    float S = sb.x;
    float Bias = sb.y;
    uint off = offsets[b];
    uint flags4 = (off & 0xFu);
    uint baseBytes = (off & ~0xFu);
    uint mode = (flags4 & 0x1u);
    uint baseU32 = baseBytes >> 2;

    for (uint k = 0; k < 16; k++) {
        uint i = lane * 16 + k;
        int codeInt = 0;

        if (mode == 0u) {
            uint word = i >> 3;
            uint sub = i & 7u;
            uint packed = codes_blob[baseU32 + word];
            uint nib = (packed >> (sub * 4u)) & 0xFu;
            codeInt = int4_to_int(int(nib));
        } else {
            uint word = i >> 4;
            uint sub = i & 15u;
            uint packed = codes_blob[baseU32 + word];
            uint sym = (packed >> (sub * 2u)) & 0x3u;

            codeInt = sym2_to_ternary(int(sym));
        }
    }
}
```

```

float w = S * float(codeInt) + Bias;
uint row = i >> 6;
uint col = i & 63u;

storeDecodedWeight(b, row, col, w);
}
}

```

9. Alternative embodiments (non-limiting)

The following variations may be implemented individually or in combination.

9.1 Different macroblock sizes and shapes

- Macroblocks may be 128 by 32, 32 by 128, 128 by 128, 32 by 32, 16 by 64, or other sizes.
- Macroblocks may be aligned with warp or wavefront sizes, cache line sizes, or matrix-multiplication tile shapes.
- A fixed payload size may be selected to match a chosen macroblock size. For example, a macroblock with K weights may have a fixed payload size of $K * 4$ bits (rounded up to a chosen alignment).

9.2 Different packing widths and vectorization

- Codes may be packed into 32-bit, 64-bit, or 128-bit words.
- FP4 codes may be stored as nibbles in uint32 or packed into larger vectors for fewer loads.
- Ternary codes may be stored as 2-bit symbols, using bitplane packing, or using a sign mask and a nonzero mask.

9.3 Different alignment and offset flag layouts

- Payload alignment may be 8, 16, 32, or 64 bytes, or a cache line size.
- The number of embedded flag bits may vary depending on alignment.
- Flags may encode mode, reserved bits, checksum indicator, sparsity hint, scaling variant, or entropy hint.
- Instead of embedding flags in offsets, flags may be stored in a separate array, bitfield, or header extension.

9.4 Additional modes

- Additional modes may include 3-, 5-, 6-, or 8-bit uniform codes.
- Additional modes may include floating-point 4-bit formats, block floating formats, or logarithmic codebooks.
- Additional modes may include sparse modes that store a mask plus nonzero values.

9.5 Alternative header semantics

- The header may store scale only with bias implied as zero.
- The header may store multiple scales (for example, per-row or per-column within a macroblock).
- The header may store an exponent and mantissa, or per-block calibration parameters.

9.6 Decode and compute fusion

- Decoding may be performed into a temporary buffer.
- Decoding may be fused with matrix multiplication by unpacking values on the fly.
- Decoding may be fused with other operations such as dequantization, activation scaling, or weight prefetch.

Claims (20 total)

The following claims are exemplary and may be revised by counsel.

Method claims

1. A computer-implemented method for representing a weight tensor of a neural network model, comprising: partitioning the weight tensor into a plurality of macroblocks each corresponding to a 64 by 64 region of the weight tensor; for each macroblock, storing a header comprising a scale and a bias for affine reconstruction of weights of the macroblock; for each macroblock, storing a payload of exactly 2048 bytes encoding quantized codes for 4096 weights of the macroblock; storing, for each macroblock, an offset value identifying a location of the 2048-byte payload in a codes blob; and decoding the quantized codes using the scale and the bias to reconstruct weights according to $w_hat = S * code + Bias$.
2. A computer-implemented method for representing a weight tensor of a neural network model, comprising: partitioning the weight tensor into a plurality of macroblocks each corresponding to a 64 by 64 region of the weight tensor; for each macroblock, storing a header comprising a scale and a bias for affine reconstruction; for each macroblock, selecting, from among a plurality of code modes, a code mode for the macroblock; for each macroblock, storing a payload of exactly 2048 bytes at a location in a codes blob, wherein the payload has the same fixed size for each macroblock independent of the selected code mode; wherein, for at least one code mode that uses fewer than 4 bits per weight, the payload includes unused or reserved bytes that are set to a predetermined value or ignored by a decoder; and decoding, for each macroblock, codes from the payload according to the selected code mode and reconstructing weights using the header.
3. The method of claim 1, wherein the header stores the scale and bias as half-precision floating-point values packed into a single 32-bit word.
4. The method of claim 1, wherein each 2048-byte payload begins at an address aligned to at least 16 bytes.

5. The method of claim 4, wherein the offset value has one or more low bits that encode a mode flag identifying how the 2048-byte payload is to be decoded.
6. The method of claim 5, wherein the mode flag is stored in the four low bits of the offset value.
7. The method of claim 5, wherein the mode flag indicates whether codes in the 2048-byte payload comprise 4-bit signed codes or ternary codes.
8. The method of claim 7, wherein when the mode flag indicates 4-bit signed codes, the 2048-byte payload stores the 4-bit signed codes as two's-complement nibbles.
9. The method of claim 8, wherein the two's-complement nibbles are packed as eight 4-bit codes per 32-bit word.
10. The method of claim 7, wherein when the mode flag indicates ternary codes, the 2048-byte payload stores ternary codes as 2-bit symbols per weight value.
11. The method of claim 10, wherein a reserved 2-bit symbol is decoded as zero.
12. The method of claim 1, further comprising selecting, for each macroblock, a candidate code mode from a plurality of candidate code modes based on an error metric computed for the macroblock.

System claims

13. A system comprising: one or more processors; and memory storing instructions that, when executed by the one or more processors, cause the system to perform the method of any of claims 1 through 12.
14. The system of claim 13, wherein one or more processors comprise a graphics processing unit configured to decode a plurality of macroblocks in parallel.
15. The system of claim 14, wherein the graphics processing unit is configured to execute a compute shader that: unpacks the scale and the bias from the header; reads the offset value; and decodes the 2048-byte payload based on the mode flag encoded in the low bits of the offset value.
16. The system of claim 13, wherein the memory stores a headers array, an offsets array, and a codes blob, and wherein the codes blob comprises concatenated 2048-byte payloads.
17. The system of claim 14, wherein a workgroup decodes at least one macroblock, and each thread decodes multiple weight values.

Non-transitory computer-readable medium claims

18. A non-transitory computer-readable medium storing a quantized representation of a weight tensor, the quantized representation comprising: a headers array storing, for each macroblock of a plurality of 64 by 64 macroblocks, a header comprising a scale and a bias; an offsets array storing, for each macroblock, an offset value, wherein each offset value identifies a location of a payload in a codes blob and includes low bits encoding a mode flag; and the codes blob storing, for each macroblock, a payload of exactly 2048 bytes for the macroblock, the payload having the same fixed size for each macroblock independent of a code mode indicated by the mode flag; wherein, for at least one code mode that uses fewer than 4 bits per weight, the payload includes reserved or unused

bytes that are set to a predetermined value or ignored by a decoder; and wherein the scale and the bias are usable to reconstruct weights according to $w_{\text{hat}} = S * \text{code} + \text{Bias}$.

19. The non-transitory computer-readable medium of claim 18, wherein the mode flag indicates whether the payload encodes 4-bit signed codes or ternary codes.
20. The non-transitory computer-readable medium of claim 18, wherein each payload is aligned to at least 16 bytes in the codes blob, and wherein the low bits used for the mode flag are low bits that are constant for aligned offsets.